Continuous Compliance Monitoring & Audit-Evidence Automation (CIS/STIG on Intune + Entra) — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 4

The problem . . . 4

The solution . . . 4

What is included . . . 4

Business value . . . 5

Bottom line . . . 5

Solution Overview — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 5

What this solution delivers . . . 5

Why GCC High is a different solution, not a re-login . . . 6

The continuous-compliance model . . . 6

Commercial vs GCC High — what's different . . . 7

The standard deployment model . . . 8

What you get . . . 8

Requirements — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 9

R1. An existing CIS/STIG baseline deployed to Intune + Entra (in the GCC High tenant) . . . 9

R2. Intune + Entra roles with Microsoft Graph **US Gov** read access . . . 10

R3. A reporting / evidence destination (in-boundary, US-sovereign) . . . 10

R4. Defined evidence-retention needs (aligned to your ATO / FedRAMP obligations) . . . 11

R5. A defined pilot scope . . . 11

Implementation Guide — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 11

Stage 0 — Prepare (once) . . . 12

Stage 1 — Assess: define the monitored control set (the baseline reference) . . . 12

Stage 2 — Configure: set scope and measurement . . . 13

Stage 3 — Pilot: measure, detect drift, export evidence . . . 13

Stage 4 — Validate: confirm outputs meet audit / ATO requirements . . . 14

Stage 5 — Expand: fleet-wide and scheduled . . . 14

Script summary (order of use) . . . 15

Testing — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 15

T0 — Correct cloud (GCC High / DoD) . . . 16

T1 — Connectivity & scope (read-only) . . . 16

T2 — Baseline reference is complete . . . 16

T3 — Control measured (current vs. baseline) . . . 16
T4 — Drift detected . . . 16
T5 — Evidence exported (audit-ready) . . . 17
T6 — Remediation closes the gap (preview → commit → re-measure) . . . 17
T7 — Scheduled fleet cadence (in-boundary) . . . 17
Troubleshooting — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 17
GCC High / DoD — wrong cloud, wrong endpoint, missing feature (check first) . . . 18
Connectivity & permissions . . . 18
Discovery (CCAUDITG-Discovery.ps1) . . . 19
Baseline reference . . . 19
Validation (CCAUDITG-Validation.ps1) . . . 19
Drift detection (Testing T4) . . . 20
Report / evidence (CCAUDITG-Report.ps1) . . . 20
Remediation (CCAUDITG-Remediation.ps1) . . . 20
Scope targeting (CCAUDITG-Set-ScopeTargeting.ps1) . . . 21
When to stop and escalate . . . 21
Best Practices — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 21
Stay in the sovereign cloud . . . 21
Evidence retention . . . 22
Cadence and drift handling . . . 22
Scoping and change control . . . 23
Audit & POA&M; response . . . 23
What to avoid . . . 23
Deliverables Summary — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High) 24
Documentation . . . 24
PowerShell scripts (Microsoft Graph SDK, US Gov endpoints) . . . 25
Named objects you will create and reuse . . . 26
Outcome . . . 26
Scope note . . . 26
Appendix A — PowerShell Scripts 27

Executive Summary — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

The problem

Passing a CIS or STIG baseline **once** is not the same as **proving** you are still compliant months later. In a **US sovereign-cloud (GCC High / DoD)** tenant this is not optional housekeeping — it is how you *keep* an authorization. Endpoints drift — a policy is edited, an exclusion is added, a device enrolls without the baseline — and when an assessor asks for evidence under FedRAMP continuous monitoring, teams scramble to gather it by hand. Point-in-time compliance and manual evidence collection are a standing audit risk.

Worse, a Gov tenant is **not the Commercial portal with a different sign-in**. The admin portals and service endpoints are different sovereign-cloud hosts (`intune.microsoft.us`, `entra.microsoft.us`, `graph.microsoft.us`), and several Commercial reporting/remediation conveniences are **not available** in GCC High. Commercial runbooks silently query the wrong cloud or call features that aren't there.

The solution

This is the **Government (GCC High)** edition: a **repeatable operational method — with PowerShell scripts you keep** — to continuously monitor baseline **drift** and automate **audit evidence** for Intune + Entra endpoints, using the **correct US Gov portals and endpoints**, with the **Commercial-vs-GCC High parity gaps called out**. It follows a simple, provable cycle:

Baseline → Measure → Detect drift → Evidence / POA&M → Remediate / track.

The scripts read the live state of your CIS/STIG controls from **Microsoft Graph US Gov** (`graph.microsoft.us`, read-only), compare it to your approved baseline, flag every control that has drifted, and export **dated audit evidence** plus a **POA&M-ready gap list** — the exact artifacts FedRAMP continuous monitoring and NIST 800-53 **CA-5** expect. Drifted settings can then be safely re-applied (preview first) and re-measured to prove closure.

This is an **operations product**, distinct from a one-time baseline build. The build creates the policies; this solution proves they stay true and produces the paperwork on demand — from inside your Gov boundary.

What is included

- A complete documentation set: Overview (with a **Commercial vs GCC High** section), Requirements,

Implementation (Gov portals + inline parity caveats), Testing (pass/fail gates, including a **correct-cloud** gate), Troubleshooting (GCC-High gotchas), Best Practices, and Deliverables — plus a cited **GCC High parity notes** reference.

- **Five PowerShell scripts** (Microsoft Graph SDK, **US Gov endpoints**): control-state discovery,

per-device baseline conformance, an audit-evidence + drift/gap report (POA&M-ready), safe remediation (preview by default), and a target-group builder (preview by default).

- A phased rollout — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with pass/fail gates and a

scheduled, in-boundary continuous-monitoring cadence.

Business value

- **Prove compliance any day**, not just deployment day — for your GCC High / DoD authorization.
- **Detect drift automatically** before an assessor does.
- **Cut audit-prep effort** — evidence and POA&M gaps export on demand, correctly formatted for FedRAMP continuous monitoring.
- **Avoid the sovereign-cloud traps** — correct `.us` portals and US Gov Graph, with parity gaps (Power BI DWH connector, ServiceNow connector, update rings, BIOS/DFCI, Remote Help) handled up front.
- **Own the capability** — a method and scripts you reuse across audit cycles, at a fraction of ongoing consulting cost.

Bottom line

Continuous, provable endpoint compliance on GCC High — drift detected and audit evidence exported on demand, POA&M-ready, in the right cloud with parity gaps handled — proven on a pilot before you scale. The specific CIS/STIG rule content is deferred to CIS and DISA; this solution delivers the precise monitoring and evidence method on Intune + Entra in the US sovereign cloud.

Solution Overview — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

What this solution delivers

This is the **Government (GCC High)** edition of the continuous-compliance solution. Passing a CIS or STIG baseline **once** is not the same as **proving** you are still compliant months later, when an assessor asks for evidence. In a **US sovereign-cloud** tenant this is not a nicety — it is how you keep an authorization. Devices drift: a policy is edited, an exclusion is added, a new device enrolls without the baseline, a user with local rights changes a setting.

This solution gives a competent-but-novice government administrator a **repeatable, operational method** — with PowerShell scripts you keep and reuse, all targeting **US Gov Graph** — to:

1. **Baseline** — take the CIS/STIG control set you already deployed to Intune + Entra in **GCC High** and treat it as the *expected state* (the reference you measure against).
1. **Measure** — read the *current state* of those controls for each in-scope device and identity directly from **Microsoft Graph US Gov** (`graph.microsoft.us`, read-only).
1. **Detect drift** — compare current state to the baseline and flag every control that no longer matches (drifted, missing, or newly non-conformant).

1. **Evidence / POA&M** — export the measurement as **audit evidence** (a dated, per-control,

per-device record) and produce a **POA&M-ready gap list** (Plan of Action & Milestones) of the controls that fail — the exact artifact FedRAMP continuous monitoring and NIST 800-53 **CA-5** expect.

1. **Remediate / track** — safely re-apply the drifted baseline assignment (preview first) and

re-measure to confirm the gap closed — then keep the cycle running on a schedule.

This is an **operations product**, distinct from a one-time baseline *build*. The build creates the policies; **this solution watches them stay true and produces the paperwork** a Gov audit requires.

Why GCC High is a different solution, not a re-login

Intune + Entra in **GCC High is not the Commercial portal with a different sign-in.** Two things differ in ways that break Commercial runbooks:

• **The portals and endpoints are different sovereign-cloud hosts.** You administer Intune at

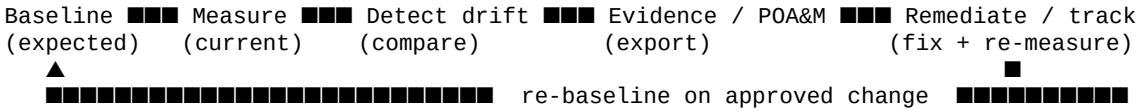
`intune.microsoft.us`, Entra at `entra.microsoft.us`, sign in at `login.microsoftonline.us`, and your scripts must target **US Gov Graph** (`graph.microsoft.us`, `Connect-MgGraph -Environment USGov`; DoD uses `dod-graph.microsoft.us`, `-Environment USGovDoD`). Point a Commercial runbook at `graph.microsoft.com` or `intune.microsoft.com` and you are querying the wrong cloud — it will silently return nothing or fail auth.

• **Feature parity lags Commercial.** Several Intune reporting, update, and remediation conveniences

that are generally available in Commercial are, in GCC High/DoD, **planned-not-yet-available** or **not available at all** (for example the Intune Power BI/Data-Warehouse connector, the ServiceNow connector, Windows Autopatch, feature/quality update rings, BIOS/DFCI profiles, Remote Help). A Commercial runbook that assumes those will silently fail: the menu or option simply is not there.

The good news for *this* solution: the **read-only measurement and evidence method uses Standard MDM state + Graph**, which **is** supported in GCC High/DoD. The parity gaps hit the **downstream analytics/integration and update tooling** — so this solution's own CSV/JSON evidence + POA&M exports (optionally backed by Log Analytics) are exactly the right pattern for a Gov tenant. This edition calls out the correct GCC High portals/endpoints at each step and flags the parity gaps **where they matter**. The full parity reference lives in `gcc-high-parity-notes.md` and is cited throughout.

The continuous-compliance model



• **Baseline (expected state).** Your approved CIS/STIG configuration in Intune (Settings Catalog /

device-configuration / compliance policies / Security Baselines) and Entra (Conditional Access, authentication methods, tenant security settings) **in the GCC High tenant**. The specific CIS Benchmark / DISA STIG **rule IDs and official control text are deferred to CIS and DISA** — this solution does not reproduce or fabricate them. (Microsoft itself points GCC High/DoD admins to the DISA STIGs library on `public.cyber.mil` for the hardening content.)

• **Measure (current state).** The scripts query **Microsoft Graph US Gov** read-only for the live

value of each monitored control on each in-scope device/identity — no changes during measurement.

- **Detect drift.** Current vs. expected, per control per device: **Pass** (matches baseline),

Drift (was compliant, now differs), **Missing** (baseline not applied to this device), or **Non-conformant** (never matched).

- **Evidence / POA&M.** Two exports: (a) an **audit-evidence** package — timestamped, immutable

CSV/JSON showing what was measured, when, by whom, and the result; (b) a **POA&M-ready gap** list — each failing control with device count, severity, and a milestone/owner column ready to paste into your organization's POA&M template. (The Intune ServiceNow connector is **not available** in GCC High, so the POA&M CSV is designed to be imported into your GRC/ITSM by hand — see `gcc-high-parity-notes.md` §2.)

- **Remediate / track.** For drifted settings only, a safe remediation re-applies the baseline

targeting (default **preview / -WhatIf** — nothing changes until you confirm). Re-measure to prove closure; the evidence trail records the fix.

Commercial vs GCC High — what's different

*(Portals are always different. Feature statuses are doc-confirmed from Microsoft's *Intune Government Service overview*, 2026-06-04, unless flagged. See `gcc-high-parity-notes.md` for the full tables and citations.)*

Portals & endpoints (always different):

Purpose	Commercial	GCC High / DoD
Sign-in	`login.microsoftonline.com`	`login.microsoftonline.us`
Intune admin center	`intune.microsoft.com`	`intune.microsoft.us`
Entra admin	`entra.microsoft.com`	`entra.microsoft.us`
Microsoft 365 admin	`admin.microsoft.com`	`portal.office365.us`
Graph (scripts)	`graph.microsoft.com` (`-Environment Global`)	`graph.microsoft.us` (`-Environment USGov`) · DoD `dod-graph.microsoft.us` (`-Environment USGovDoD`)

Feature parity gaps that change how you design (GCC High/DoD status in parentheses):

- **Intune Power BI connector for Data Warehouse** is **not available** (*no planning*). Do **not**

build evidence dashboards on the Intune DWH/Power BI connector; use this solution's Graph exports plus **Log Analytics** (which *is* available).

- **ServiceNow connector** is **not available** (*no planning*). Import the POA&M CSV into your

GRC/ITSM manually rather than pushing via the native connector.

- **Windows Autopatch** and **feature / quality / expedite / driver update rings for Windows** are

planned — not yet available. If a control depends on a patch level, drive updates through your existing Gov-approved mechanism, not these Intune rings.

• **BIOS configuration profiles / DFCI management** are **planned — not yet available**. STIG rules needing BIOS/DFCI enforcement are out-of-band in GCC High for now — flag them.

• **Remote Help** is **planned — not yet available** (no native Intune hands-on remediation tool).

• **Cloud PKI** is **planned — GCC High only** (not listed for **DoD**). Verify for DoD if certificate controls are in your baseline.

• **Security Copilot in Intune** and **Windows Device Health Attestation (DHA)** are **planned — not yet available**. Do not assume Copilot triage or DHA as a compliance signal.

• **Advanced Analytics**, **Log Analytics/Azure Monitor export**, and **Standard MDM** (compliance policies, configuration/Settings Catalog profiles, device compliance state) **are available** — this is what the solution reads. **Verify** the specific analytics report you want renders in `intune.microsoft.us`.

Where an option is not visible in `intune.microsoft.us`, treat it as a **GCC High/DoD parity gap, not a permissions problem** — re-check the parity table before troubleshooting.

The standard deployment model

This solution follows the category standard, with a **pass/fail gate** at each stage:

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

• **Assess** — confirm Gov licensing (G3/G5 / EMS G-SKUs), roles, **GCC High portal access**, and which features are actually available in your GCC High/DoD tenant (the parity check).

• **Configure** — turn your deployed CIS/STIG baseline into a machine-readable expected state; build the scope groups. Name every object verbatim.

• **Pilot** — prove the method on a small **named pilot group** (`CCAUDITG-Pilot-Devices`): measure, seed a controlled drift, export evidence.

• **Validate** — run the pass/fail gates against the pilot; confirm evidence and POA&M meet your ATO requirements.

• **Expand** — widen to the fleet on a schedule only after the gate passes.

The scripts are **read-only by default**; the only script that changes anything (Remediation) and the targeting builder (Set-ScopeTargeting) run in **preview (-WhatIf)** until you explicitly commit.

What you get

• This documentation set: **Overview**, **Requirements**, **Implementation** (Gov portals + inline parity caveats; drift monitoring + POA&M evidence), **Testing** (pass/fail gates), **Troubleshooting** (with GCC-High-specific gotchas), **Best Practices**, **Deliverables Summary**, and **Executive Summary**.

• A **GCC High parity notes** reference (`gcc-high-parity-notes.md`) with per-feature status and Microsoft citations.

• **Five PowerShell scripts** (Microsoft Graph SDK, **US Gov endpoints**): control-state **Discovery**,

per-device baseline **Validation**, an audit-evidence + drift/gap **Report** (POA&M-ready), safe **Remediation** of drifted settings (default preview), and a **Set-ScopeTargeting** pilot/fleet-group builder (default preview).

> ***Sources:*** *Microsoft Learn — *Microsoft Intune Government Service overview* (2026-06-04); *Network endpoints for US government deployments — Microsoft Intune* (2025-12-15); *Access national cloud deployments with the Microsoft Graph SDKs*; *Microsoft Entra feature availability in Azure for US Government*. Full citations in* `gcc-high-parity-notes.md`.

Requirements — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

Confirm each prerequisite below **before** you begin the Implementation Guide. Category-level prerequisites (Entra tenant, Intune subscription, admin roles, enrollment, pilot group, supported OS, network, change control) are listed once in **global-requirements-security.md** — read that first. The items here are **specific to continuous compliance monitoring and audit-evidence automation on a GCC High tenant**. Each entry follows: **Description / Why / Verify / Implement if missing.**

> ***GCC High note:*** *every "Verify" below uses the **US sovereign-cloud portals** —* `intune.microsoft.us`*,* `entra.microsoft.us`*,* `portal.office365.us`*, sign-in at* `login.microsoftonline.us`*. If any portal loads at a* `.com` *host you are in the **Commercial** cloud — stop and switch. Scripts connect to **US Gov Graph** (*`graph.microsoft.us`*,* `Connect-MgGraph -Environment USGov`*; DoD* `dod-graph.microsoft.us`*,* `-Environment USGovDoD`*).*

R1. An existing CIS/STIG baseline deployed to Intune + Entra (in the GCC High tenant)

Description: A CIS Benchmark and/or DISA STIG configuration already built and assigned in Intune (Settings Catalog / device-configuration / compliance policies / Security Baselines) and Entra (Conditional Access, authentication methods, tenant settings) **in your GCC High tenant** — the *expected state* you will measure against.

Why: This solution measures **drift from a baseline**. Without a defined baseline there is nothing to compare current state to, no "expected value" for evidence, and no gap to put in a POA&M. The specific CIS/STIG rule IDs and official control text come from **CIS and DISA** — this solution does not supply them. (Microsoft points GCC High/DoD admins to the DISA STIGs Document Library on `public.cyber.mil` for the hardening content.)

Verify: In `https://intune.microsoft.us` → **Endpoint security** → **Security baselines** and/or **Devices** → **Manage devices** → **Configuration**, confirm your baseline profiles exist and are assigned. In `https://entra.microsoft.us` → **Entra ID** → **Conditional Access** confirm the identity controls exist. You should be able to name the exact objects that encode your baseline.

Implement if missing: Build the one-time CIS/STIG baseline first (a separate build activity), in the **GCC High** tenant: import the CIS/STIG content into Intune Settings Catalog / Security Baselines, create the matching Entra Conditional Access and authentication-method policies, and assign them. Then return here to monitor it. **Parity check:** confirm each control you intend to monitor is enforced by a mechanism that **exists in GCC High** — e.g. STIG rules requiring **BIOS/DFCI** enforcement or **Windows update rings** are **not yet available** in GCC High (see `gcc-high-parity-notes.md` §3) and must be handled

out-of-band.

R2. Intune + Entra roles with Microsoft Graph US Gov read access

Description: An operator identity holding read rights to Intune and Entra configuration and device state in the GCC High tenant, and consented Microsoft Graph **US Gov read scopes** for the monitoring scripts.

Why: Discovery, Validation, and Report are **read-only** and must be able to read policies, compliance state, Conditional Access, and directory objects via **US Gov Graph**. Insufficient scopes cause empty or partial evidence — an audit gap. Connecting to the wrong cloud (Commercial Graph) returns nothing.

Verify: In `https://entra.microsoft.us` → **Roles and administrators**, confirm the operator has at least a read role (for example **Global Reader** plus **Intune Administrator** for later remediation). Run `CCAUDITG-Discovery.ps1` (read-only) and confirm it connects to **US Gov** and returns data without a permissions error. Confirm the connected environment with `Get-MgContext` (Graph host should be `graph.microsoft.us`).

Implement if missing: Assign **Global Reader** (directory/CA read) and **Intune Administrator** (Intune read/write for remediation). On first script run, consent the required delegated Graph read scopes when prompted: `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Policy.Read.All`, `Directory.Read.All`, `Group.Read.All`. (Remediation additionally needs `DeviceManagementConfiguration.ReadWrite.All`; Set-ScopeTargeting needs `Group.ReadWrite.All`.) The scripts pass `-Environment USGov` (use `-Environment USGovDoD` on a DoD/IL5 tenant) so consent happens against the **US Gov** identity endpoint.

R3. A reporting / evidence destination (in-boundary, US-sovereign)

Description: A controlled location where the exported audit-evidence and POA&M-ready gap files are written and stored — for example a restricted, access-logged **SharePoint (GCC High)** document library or a US Gov file share the scripts' output folder points to. Optionally, a **Log Analytics / Azure Monitor (Azure Government)** workspace as a durable, queryable evidence store.

Why: Audit evidence has value only if it is stored somewhere durable, access-controlled, tamper-evident, and **inside your authorization boundary**. Writing evidence to a random local folder — or worse, a Commercial-cloud location — undermines its integrity and can break data-residency.

Verify: Confirm you have a path (local staging + a controlled, in-boundary destination) where the operator can write, that is backed up, access-restricted, and retained per policy. Confirm you can write a test file there. If using Log Analytics, confirm the workspace is in **Azure Government**.

Implement if missing: Create a dedicated folder / library (for example a GCC High SharePoint document library or restricted network share) with read/write limited to the compliance operators, versioning or immutability enabled where available, and backup configured. Point the scripts' `-OutputPath` at your local staging folder and copy finished evidence to the controlled destination. The Intune **Log Analytics/Azure Monitor export is available** in GCC High/DoD if you want a queryable store; the **Power BI Data-Warehouse connector is not** (see `gcc-high-parity-notes.md` §2).

R4. Defined evidence-retention needs (aligned to your ATO / FedRAMP obligations)

Description: A written statement of how long audit evidence must be kept, in what format, and who may access it — derived from your authorization (FedRAMP High / DoD IL4–IL5), regulatory framework, and continuous-monitoring cadence.

Why: Federally regulated environments mandate retention periods and a **monthly** continuous- monitoring / POA&M cadence; the export cadence, storage, and disposal must match. Retention that is too short fails audits; unmanaged retention creates risk. NIST 800-53 **CA-5** requires a maintained POA&M as the record of open weaknesses.

Verify: Confirm a retention requirement exists (period, format, access, disposal) and that your destination (R3) enforces it. Confirm ownership of the retention decision and the monitoring cadence your ATO expects (commonly monthly for FedRAMP High).

Implement if missing: Document the retention requirement with your compliance/ISSO/AO (period, immutable vs. versioned, access list, disposal method) and the monitoring cadence. Configure the destination's retention/hold settings to match, and record the cadence (for example measure at least monthly, retain evidence per policy). **Verify the exact obligations against your own authorization boundary and ATO** — the timelines cited in `gcc-high-parity-notes.md` §5 are the general FedRAMP framework, not a substitute for your assessor's determination.

R5. A defined pilot scope

Description: A small, representative Microsoft Entra security group of devices (and/or users) used to prove the monitoring method before fleet-wide rollout — named `CCAUDITG-Pilot-Devices`.

Why: The category standard is Assess → Configure → **Pilot** → Validate → Expand. Piloting on a small group first lets you validate measurement accuracy, drift detection, and evidence output without noise or risk across the whole fleet, and lets you safely seed a test drift.

Verify: In `https://entra.microsoft.us` → **Groups**, confirm `CCAUDITG-Pilot-Devices` exists and contains a small set of representative in-scope devices (mix of OS builds / roles).

Implement if missing: Create it — `entra.microsoft.us` → **Groups** → **New group** → type **Security**, name `CCAUDITG-Pilot-Devices`, add a handful of representative enrolled devices — or generate it from your baseline scope with `CCAUDITG-Set-ScopeTargeting.ps1` (default preview / -WhatIf; review, then commit).

When R1–R5 and the global requirements are all satisfied, proceed to the Implementation Guide.

Implementation Guide — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

This guide gives you the **exact, ordered steps** to stand up continuous compliance monitoring and audit-evidence automation for your CIS/STIG baseline on Intune + Entra **in a GCC High tenant**. Follow the category deployment model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a pass/fail

gate at each stage. All discovery/measurement is **read-only**; the only changing actions (Remediation, Set-ScopeTargeting) default to **preview (-WhatIf)**.

Everything uses the US sovereign-cloud portals — sign in at `https://login.microsoftonline.us`, administer Intune at `https://intune.microsoft.us` and Entra at `https://entra.microsoft.us`. If a portal loads at a `.com` host, **stop — you are in the Commercial cloud.** The scripts connect to **US Gov Graph** (`graph.microsoft.us`, `-Environment` USGov; DoD/IL5 tenants use `dod-graph.microsoft.us`, `-Environment USGovDoD`).

Naming — create and reuse these object names verbatim (in later steps and in the scripts):

Object	Name to create and reuse
Pilot Entra security group	`CCAUDITG-Pilot-Devices`
Fleet Entra security group	`CCAUDITG-Fleet-Devices`
Baseline reference file	`CCAUDITG-Baseline.json`
Evidence exports	`CCAUDITG-Evidence-<timestamp>.csv/json`
POA&M exports	`CCAUDITG-POAM-<timestamp>.csv`

GCC High parity reminders are inlined at the step where they matter. Full detail: `gcc-high-parity-notes.md`. Work through the stages in order. **Do not skip the pilot.**

Stage 0 — Prepare (once)

1. Confirm every item in **02-requirements.md** and **global-requirements-security.md** is satisfied.
2. Install the Microsoft Graph PowerShell SDK on the operator workstation (elevated PowerShell):

```
Install-Module Microsoft.Graph -Scope CurrentUser
```

1. Copy the five **CCAUDITG-*.ps1** scripts to a working folder, for example `C:\CCAUDITG\scripts`,

and create an output folder `C:\CCAUDITG\out` (your local staging → copy finished evidence to the controlled, **in-boundary** destination from R3).

1. Sign in once to confirm scopes **against US Gov**:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Discovery.ps1 -OutputPath C:\CCAUDITG\out
```

(The script passes `-Environment USGov` by default; add `-Environment USGovDoD` on a DoD tenant.) Consent the read scopes when prompted at `login.microsoftonline.us`. **Gate 0 (pass):** the script connects to **US Gov Graph** (`Get-MgContext` shows a `graph.microsoft.us` host) and writes a discovery file with no permissions error.

> ***Parity caveat:*** *if the connect lands on `graph.microsoft.com`, you are in the Commercial cloud — re-run with the correct `-Environment` value. This is the single most common GCC High mistake.*

Stage 1 — Assess: define the monitored control set (the baseline reference)

You are turning your deployed **GCC High** CIS/STIG baseline into a **machine-readable expected state** the scripts measure against.

1. **Discover current configuration.** Run control-state discovery to inventory the Intune and Entra

objects that encode your baseline:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Discovery.ps1 -OutputPath C:\CCAUDITG\out -EmitBase
```

It writes `CCAUDITG-Discovery-<timestamp>.csv/json` listing the discovered configuration profiles, compliance policies, Security Baselines, and Entra Conditional Access / authentication-method policies with their current settings, plus a `CCAUDITG-Baseline.template.json` to complete.

1. **Select the monitored controls.** From the discovery output, choose the controls that make up your

monitored set — the CIS/STIG settings you must prove. Record, for each control: a stable **control identifier** (use your CIS/STIG reference verbatim — deferred to CIS/DISA), the **Intune or Entra object** that carries it, the setting name, and the **expected value**.

> ***Parity caveat:*** *only monitor controls whose enforcement mechanism* ***exists in GCC High****. Do* ***not*** *add controls that depend on* ***BIOS/DFCI profiles****,* ***Windows Autopatch / update rings****,* ***Device Health Attestation****, or* ***Remote Help*** *— these are* ***planned-not-yet-available*** *in GCC High (`gcc-high-parity-notes.md` §3). Manage those out-of-band and mark them as such in your POA&M.*

1. **Create the baseline reference file.** Save the selected controls as `CCAUDITG-Baseline.json`

in the working folder — this is your expected-state reference. Use the template the Discovery script emits so field names match what Validation reads. Store it under change control (R1/G8).

Gate 1 (pass): `CCAUDITG-Baseline.json` exists, each entry has a control ID, target object, setting, and expected value, and it is approved by your compliance lead / ISSO.

Stage 2 — Configure: set scope and measurement

1. **Build the pilot target group.** If `CCAUDITG-Pilot-Devices` does not exist, create it —

preview first, then commit:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Set-ScopeTargeting.ps1 -GroupName CCAUDITG-Pilot-De
```

Review the previewed membership (default -WhatIf). Re-run with `-Commit` to create/populate it. (The script connects to US Gov Graph by default.)

1. **Dry-run measurement against the baseline.** Validate current vs. expected for the pilot group:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Validation.ps1 -BaselinePath .\CCAUDITG-Baseline.js
```

It writes a per-device, per-control conformance file with verdicts (Pass / Drift / Missing / Non-conformant).

Gate 2 (pass): every monitored control is measured for every pilot device (no "not measured" rows caused by scope or scope-tag gaps).

Stage 3 — Pilot: measure, detect drift, export evidence

1. **Baseline measurement.** Run Validation against the pilot (as above) and keep the output as the

pilot **baseline measurement**.

1. **Seed a controlled drift (test).** In a change window, deliberately change one non-critical

monitored setting on one pilot device / one pilot-scoped policy (record the change). This proves detection works. Do **not** seed authentication-impacting drift (G8).

1. **Detect the drift.** Re-run Validation. Confirm the seeded change appears as **Drift** for the

expected control/device, and unchanged controls remain **Pass**.

1. **Export audit evidence + POA&M gaps.** Generate the evidence and gap exports:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Report.ps1 -ValidationPath C:\CCAUDITG\out\CCAUDITG
```

It writes: `CCAUDITG-Evidence-<timestamp>.csv/json` (dated per-control per-device evidence with tenant, operator, expected, observed, verdict) and `CCAUDITG-POAM-<timestamp>.csv` (one row per failing control with device count, severity, and empty milestone/owner columns to complete).

> ***Parity caveat:*** *the Intune* ***ServiceNow connector is not available*** *in GCC High (*`gcc-high-parity-notes.md` *§2). Import* `CCAUDITG-POAM-<timestamp>.csv` *into your GRC/ITSM (or your FedRAMP POA&M workbook)* ***manually*** *— the CSV is shaped for that. Likewise, build any dashboard on* ***Log Analytics****, not the (unavailable) Intune Power BI/DWH connector.*

1. **Remediate the seeded drift (preview then commit).** Preview the fix, then commit:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Remediation.ps1 -ValidationPath C:\CCAUDITG\out\CCA
```

Default is -WhatIf (preview). Review the planned re-application of baseline targeting for the drifted control, then re-run with `-Commit`. Re-run Validation to confirm the control returns to **Pass**.

Gate 3 (pass): the seeded drift was detected, evidence exported with correct fields, POA&M row generated, and remediation returned the control to Pass on re-measure. Copy the evidence to the controlled, in-boundary destination (R3).

Stage 4 — Validate: confirm outputs meet audit / ATO requirements

1. Have your compliance lead / ISSO review a sample evidence export and confirm the fields, format, and

retention match your **FedRAMP High / DoD IL** audit expectations (R4).

1. Confirm the POA&M export maps cleanly into your organization's POA&M template / FedRAMP POA&M

workbook (control ID, weakness description, affected devices, severity, milestones, owner). Confirm the remediation-timeline columns align to your obligations (commonly 30/90/180 days for High/Moderate/Low — **verify against your ATO**, `gcc-high-parity-notes.md` §5).

1. Confirm evidence lands in the controlled, in-boundary destination and is retained per policy.

Gate 4 (pass): audit lead / ISSO accepts the evidence and POA&M formats; retention and cadence confirmed.

Stage 5 — Expand: fleet-wide and scheduled

1. **Broaden scope.** Point Validation/Report at your full in-scope set (a fleet group such as

`CCAUDITG-Fleet-Devices`, built with Set-ScopeTargeting) instead of the pilot group.

1. **Schedule the cycle.** Create a scheduled task on the operator workstation (or a managed runbook host **inside your Gov boundary**) that runs, on your continuous-monitoring cadence (for example **monthly** for FedRAMP High, or weekly): Discovery → Validation → Report, writing to staging and copying evidence to the controlled destination.

> ***Parity caveat:*** *run the schedule on a host inside your authorization boundary that reaches* ***US Gov endpoints*** *(`graph.microsoft.us`, `login.microsoftonline.us`). The scheduled account must have consented the* ***US Gov*** *scopes interactively once.*

1. **Route drift.** Treat each new **Drift** / **Missing** / **Non-conformant** row as an event: open a POA&M item, remediate under change control (Remediation preview → commit), and re-measure to close.

1. **Re-baseline on approved change.** When your CIS/STIG baseline legitimately changes (approved), re-run Discovery, update `CCAUDITG-Baseline.json`, and re-approve it — so expected state stays accurate.

Gate 5 (pass): the scheduled cycle runs unattended, evidence accumulates in the controlled destination on cadence, and drift is tracked to closure via POA&M.

Script summary (order of use)

Order	Script	Purpose	Changes anything?
1	`CCAUDITG-Discovery.ps1`	Inventory control state (US Gov Graph); emit baseline template	No (read-only)
2	`CCAUDITG-Set-ScopeTargeting.ps1`	Build pilot/fleet target groups	Preview by default (-WhatIf)
3	`CCAUDITG-Validation.ps1`	Per-device baseline conformance (drift verdicts)	No (read-only)
4	`CCAUDITG-Report.ps1`	Audit evidence + POA&M-ready gap export	No (read-only)
5	`CCAUDITG-Remediation.ps1`	Safe fix of drifted settings	Preview by default (-WhatIf)

All scripts default to `-Environment USGov`; pass `-Environment USGovDoD` on a DoD/IL5 tenant.

Proceed to **05-testing.md** for the formal pass/fail gates and **06-troubleshooting.md** if a step does not behave as described.

Testing — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

Use these **pass/fail gates** to prove the solution works before you rely on it for a Gov audit. The three core outcomes you must demonstrate are: **a control is measured**, **drift is detected**, and **evidence is exported** — all against the **US sovereign cloud**. Run the gates on the pilot group (`CCAUDITG-Pilot-Devices`) first, then confirm the fleet cadence.

Record the result of each gate (Pass / Fail), the date, and the operator. Do not expand to the fleet until Gates T1–T6 pass.

T0 — Correct cloud (GCC High / DoD)

Test: Run `CCAUDITG-Discovery.ps1 -OutputPath C:\CCAUDITG\out` and then `Get-MgContext`. **Pass:** The connection is against **US Gov** — the Graph host is `graph.microsoft.us` (GCC High) or `dod-graph.microsoft.us` (DoD), and sign-in was at `login.microsoftonline.us`. **Fail:** The context shows `graph.microsoft.com` (Commercial) → re-run with the correct `-Environment USGov` / `-Environment USGovDoD`; you were querying the wrong cloud.

T1 — Connectivity & scope (read-only)

Test: Run `CCAUDITG-Discovery.ps1 -OutputPath C:\CCAUDITG\out`. **Pass:** Script connects to US Gov Graph with the read scopes, completes without a permissions error, and writes `CCAUDITG-Discovery-<timestamp>.csv/json` containing your baseline's Intune profiles, compliance policies, Security Baselines, and Entra Conditional Access / auth-method policies. **Fail:** Permissions error, empty output, or missing baseline objects → see R2 and Troubleshooting.

T2 — Baseline reference is complete

Test: Open `CCAUDITG-Baseline.json`. **Pass:** Every monitored control has a stable **control ID**, a **target object** (Intune/Entra), the **setting name**, and an **expected value**; the file is approved under change control. No control depends on a mechanism that is **unavailable in GCC High** (BIOS/DFCI, update rings, DHA — see `gcc-high-parity-notes.md` §3). **Fail:** Any control missing an expected value or target object, or a control tied to an unavailable GCC High mechanism → re-run Discovery with `-EmitBaselineTemplate`, complete the entry, or move the out-of-band control to your manual POA&M.

T3 — Control measured (current vs. baseline)

Test: Run `CCAUDITG-Validation.ps1 -BaselinePath .\CCAUDITG-Baseline.json -GroupName CCAUDITG-Pilot-Devices -OutputPath C:\CCAUDITG\out`. **Pass:** Every monitored control is measured for **every** pilot device — each row has a verdict of **Pass / Drift / Missing / Non-conformant** and an observed value. No "not measured" rows. **Fail:** Controls or devices missing from the output, or verdicts blank → check group membership, scope tags, and Graph read scopes (R2, R5, Troubleshooting).

T4 — Drift detected

Test: In a change window, deliberately change one non-critical monitored setting on one pilot device or pilot-scoped policy (record it). Re-run `CCAUDITG-Validation.ps1`. **Pass:** The changed control shows **Drift** for the expected device with the new observed value; all untouched controls remain **Pass**. Detection is specific (only the seeded control flips). **Fail:** The change is not detected, or unrelated controls flip → verify you edited a *monitored* control and that Validation reads the same object; see Troubleshooting.

T5 — Evidence exported (audit-ready)

Test: Run `CCAUDITG-Report.ps1 -ValidationPath <latest validation json> -OutputPath C:\CCAUDITG\out`. **Pass:** `CCAUDITG-Evidence-<timestamp>.csv/json` is written and each record contains **collection timestamp, tenant, operator identity, control ID, target object, expected value, observed value, and verdict**. `CCAUDITG-POAM-<timestamp>.csv` is written with **one row per failing control** including device count, severity, and empty **milestone/owner** columns. Files copy successfully to the controlled, in-boundary destination (R3). **Fail:** Missing fields, no POA&M rows for known failures, or destination write fails → check R3/R4 and Troubleshooting.

T6 — Remediation closes the gap (preview → commit → re-measure)

Test: Run `CCAUDITG-Remediation.ps1 -ValidationPath <latest validation json>` (default -WhatIf) and review the preview; re-run with `-Commit`; then re-run `CCAUDITG-Validation.ps1`. **Pass:** Preview lists the exact drifted control to be re-applied and changes nothing; after `-Commit`, re-measurement shows the control back to **Pass**; the evidence trail records the fix. **Fail:** Preview changes state, commit does not close the gap, or re-measure still shows Drift → do not use Remediation on the fleet; investigate per Troubleshooting.

T7 — Scheduled fleet cadence (in-boundary)

Test: After Gates T1–T6 pass, point Validation/Report at the fleet group (`CCAUDITG-Fleet-Devices`) and run the scheduled Discovery → Validation → Report cycle once unattended on a host inside your Gov boundary. **Pass:** The cycle completes on schedule at your continuous-monitoring cadence (for example monthly), connects to **US Gov Graph** unattended, evidence lands in the controlled destination, and any drift appears as POA&M rows. **Fail:** Task does not run unattended, connects to the wrong cloud, or evidence does not accumulate → check the scheduled-task account's **US Gov** Graph consent and destination permissions.

Overall pass criteria: T0–T6 pass on the pilot and T7 passes on the fleet. Only then treat the solution as production audit evidence for your GCC High / DoD authorization.

Troubleshooting — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

Use this guide when a step in the Implementation or Testing guide does not behave as described. Each entry is **Symptom** → **Likely cause** → **Fix.** Items map to the scripts and pass/fail gates. **The first section**

is GCC-High-specific — check it first, because most Gov failures are cloud/parity issues, not logic bugs.

GCC High / DoD — wrong cloud, wrong endpoint, missing feature (check first)

Symptom: A script connects but returns **no data**, or authentication fails with an unknown-account error. **Likely cause:** You connected to the **Commercial** cloud (`graph.microsoft.com`) instead of US Gov, so your Gov objects aren't there — or the account isn't recognized on the Commercial endpoint. **Fix:** Confirm the environment. Run `Get-MgContext` — the Graph host must be `graph.microsoft.us` (GCC High) or `dod-graph.microsoft.us` (DoD). Re-run the script with `-Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD). Sign-in must be at `login.microsoftonline.us`. See `gcc-high-parity-notes.md` §1.

Symptom: A portal you were told to open loads at a `.com` address. **Likely cause:** You opened the Commercial portal. **Fix:** Use the **US Gov** hosts: Intune `intune.microsoft.us`, Entra `entra.microsoft.us`, Microsoft 365 `portal.office365.us`, sign-in `login.microsoftonline.us`. If it still redirects to `.com`, your browser session is signed into a Commercial tenant — sign out and sign in to the Gov tenant.

Symptom: A menu, report, or connector referenced in a Commercial runbook is **not present** in `intune.microsoft.us` (for example the **Power BI / Data Warehouse connector**, **ServiceNow connector**, **Windows Autopatch / update rings**, **BIOS/DFCI profiles**, **Remote Help**, **Security Copilot**, **Device Health Attestation**). **Likely cause:** This is a **GCC High/DoD parity gap**, not a permissions or configuration problem — the feature is *planned-not-yet-available* or *not available* in the government cloud. **Fix:** Confirm the feature's status in `gcc-high-parity-notes.md` (§2–§4). Do not troubleshoot a missing option as if it were a bug. Use the documented alternative — export the **POA&M CSV** and import it into your GRC/ITSM manually (no ServiceNow connector), build dashboards on **Log Analytics** (no Power BI DWH connector), and manage updates/BIOS/attestation controls **out-of-band**.

Symptom: Scheduled unattended run works interactively but fails as a task. **Likely cause:** The task account never consented the **US Gov** Graph scopes, or the host cannot reach `graph.microsoft.us` / `login.microsoftonline.us`. **Fix:** Run each script once interactively **as the task account** with the correct `-Environment` to complete US Gov consent; ensure the host (inside your Gov boundary) can reach the US Gov endpoints through the proxy/firewall (see global requirement G7 and `gcc-high-parity-notes.md` §1).

Connectivity & permissions

Symptom: A script fails on connect with a consent or permissions error, or returns no data. **Likely cause:** The operator lacks a required role, or the delegated Graph read scopes were not consented (against US Gov). **Fix:** Confirm the operator holds at least **Global Reader** plus **Intune Administrator** (R2, G3). Re-run (with `-Environment USGov/USGovDoD`) and consent the read scopes when prompted: `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Policy.Read.All`, `Directory.Read.All`, `Group.Read.All`. If a tenant admin must grant consent, have them approve the app's delegated permissions in the **Gov** tenant, then re-run.

Symptom: `Microsoft.Graph module not found`. **Likely cause:** The Graph PowerShell SDK is not installed. **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`, then re-run.

Discovery (CCAUDITG-Discovery.ps1)

Symptom: Discovery runs but your baseline profiles/policies are missing from the output. **Likely cause:** The baseline was never built/assigned in the **Gov** tenant, or lives in an object type not queried, or you connected to the wrong cloud. **Fix:** Confirm `Get-MgContext` shows a `*.microsoft.us` Graph host. Confirm the baseline exists in Intune (`intune.microsoft.us` → Endpoint security → Security baselines; Devices → Configuration) and Entra (`entra.microsoft.us` → Conditional Access) — see R1. If custom objects are used, note their type; you can still record them manually in `CCAUDITG-Baseline.json`.

Symptom: Baseline template has placeholder values (`REPLACE-...`). **Likely cause:** The template is meant to be completed by you. **Fix:** Fill in each control's **ControlId** (your CIS/STIG reference — deferred to CIS/DISA), **SettingName**, **ExpectedValue**, and **Severity**, save as `CCAUDITG-Baseline.json`, and approve it under change control (Gate 1/T2). Do not include controls whose enforcement mechanism is unavailable in GCC High (BIOS/DFCI, update rings, DHA) — mark those out-of-band.

Baseline reference

Symptom: Validation throws "Baseline reference not found" or "is empty". **Likely cause:** `CCAUDITG-Baseline.json` is missing, mis-pathed, or invalid JSON. **Fix:** Point `-BaselinePath` at the correct file; validate the JSON parses (`Get-Content ... | ConvertFrom-Json`). Regenerate from Discovery `-EmitBaselineTemplate` if needed.

Validation (CCAUDITG-Validation.ps1)

Symptom: "Target group not found." **Likely cause:** `CCAUDITG-Pilot-Devices` (or the named group) does not exist in the Gov tenant. **Fix:** Create it via `entra.microsoft.us` → Groups, or run `CCAUDITG-Set-ScopeTargeting.ps1` (preview then `-Commit`). Confirm the display name matches exactly (R5).

Symptom: No device rows / "No device members found." **Likely cause:** The target group has no device members, or contains users instead of devices. **Fix:** Add the representative **enrolled devices** to the group. For device-based measurement, the group members must be devices with an `azureADDeviceId`.

Symptom: Controls show verdict **Missing** for devices you know are compliant. **Likely cause:** The device is not returned in managed-device state (not enrolled/reporting), or the baseline object is not assigned to that device. **Fix:** Confirm the device is enrolled and compliant in `intune.microsoft.us` → Devices → All devices (G4). Confirm the baseline is assigned to a group that includes the device. "Missing" is a targeting/enrollment signal, not a drift.

Symptom: Every control shows the same verdict for a device regardless of setting. **Likely cause:** Observed value is derived from overall device compliance where per-setting reporting is unavailable, so all controls share the device's compliance signal. **Fix:** This is expected where Intune does not expose per-setting state for that object type; refine `CCAUDITG-Baseline.json` to point each control at an object whose state is individually reportable, or accept device-level conformance as the observed signal for that control.

Drift detection (Testing T4)

Symptom: A deliberately-changed setting is **not** detected as Drift. **Likely cause:** You changed a setting that is not in the monitored set, or Validation reads a different object than the one you edited. **Fix:** Confirm the changed setting corresponds to a control in `CCAUDITG-Baseline.json` and that its **TargetObject** is the object you edited. Allow time for the device to report the new state, then re-run.

Symptom: Unrelated controls flip to Drift after one change. **Likely cause:** Shared observed-value signal (device compliance) affecting multiple controls, or a broad policy change. **Fix:** Narrow the baseline entries to individually-reportable objects; verify only the intended policy was changed.

Report / evidence (CCAUDITG-Report.ps1)

Symptom: "Validation file not found" or "is empty." **Likely cause:** No validation run was produced, or the wrong path/timestamp was passed. **Fix:** Run `CCAUDITG-Validation.ps1` first; pass the exact `CCAUDITG-Validation-<timestamp>.json`.

Symptom: Evidence rows are missing provenance fields (T5). **Likely cause:** The validation output was hand-edited or truncated. **Fix:** Do not edit validation output. Re-run Validation to regenerate a complete file, then re-report.

Symptom: No POA&M rows despite known failures. **Likely cause:** No rows carry Drift/Missing/Non-conformant verdicts (all Pass), or the validation file is stale. **Fix:** Confirm failures exist in the validation output; re-run Validation against current state.

Symptom: Evidence cannot be copied to the destination, or can't reach ServiceNow. **Likely cause:** Missing write permission or wrong destination path (R3) — or you expected the Intune **ServiceNow connector**, which is **not available** in GCC High. **Fix:** Grant the operator write access; verify the in-boundary destination path; confirm retention/immutability settings (R4) do not block new-file writes. For POA&M routing, import the CSV into your GRC/ITSM manually (see `gcc-high-parity-notes.md` §2).

Remediation (CCAUDITG-Remediation.ps1)

Symptom: Preview appears to change state. **Likely cause:** `-Commit` was passed (which disables preview). **Fix:** Run without `-Commit` for preview. Only add `-Commit` inside an approved change window with a documented rollback (G8).

Symptom: After `-Commit`, the control does not return to Pass on re-measure. **Likely cause:** The re-assignment did not reach the device yet, the drift source persists (for example a local admin re-changing the setting), or the control's enforcement mechanism is **not available in GCC High** (update ring, BIOS/DFCI, DHA). **Fix:** Allow policy sync time and re-run Validation. If it recurs, address the root cause (remove local rights, correct the policy). If the control relies on an unavailable GCC High mechanism, remediate **out-of-band** and record it in the POA&M rather than expecting Intune to enforce it.

Symptom: "Baseline group not found." **Likely cause:** `CCAUDITG-Fleet-Devices` (or the named baseline group) does not exist. **Fix:** Build it with `CCAUDITG-Set-ScopeTargeting.ps1` (preview then `-Commit`).

Scope targeting (CCAUDITG-Set-ScopeTargeting.ps1)

Symptom: Group/members not created after `-Commit`. **Likely cause:** The operator lacks `Group.ReadWrite.All`, or candidate objects were not devices. **Fix:** Consent `Group.ReadWrite.All` (against US Gov); confirm candidates are device objects. Re-run preview to verify the plan, then `-Commit`.

Symptom: Pilot group has too many/few members. **Likely cause:** `-PilotSampleSize` default, or a source group with unexpected membership. **Fix:** Set `-PilotSampleSize` to the desired count, or pass `-SourceGroupName` to seed from a known group; review the preview plan before committing.

When to stop and escalate

- Do **not** trust any result until `Get-MgContext` confirms you are on **US Gov** (`*.microsoft.us`).
- Do **not** run Remediation on the fleet if the pilot gates (T4/T6) did not pass.
- Do **not** treat exports as audit evidence until T0–T6 pass on the pilot and T7 on the fleet.
- Do **not** troubleshoot a missing GCC High feature as a bug — confirm it against `gcc-high-parity-notes.md` and use the documented alternative.
- If evidence integrity is ever in doubt (edited files, gaps in the series), re-run the full cycle to regenerate authoritative records rather than patching output by hand.

Best Practices — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

These practices keep your continuous-compliance program **defensible, low-risk, and audit-ready** over time on a **US sovereign-cloud (GCC High / DoD)** tenant. They focus on the areas assessors and Authorizing Officials scrutinize most: **staying in the right cloud**, **evidence retention**, and **audit / POA&M response**.

Stay in the sovereign cloud

- **Always confirm US Gov before you trust a result.** Every scheduled and interactive run should verify the Graph host is `graph.microsoft.us` (GCC High) or `dod-graph.microsoft.us` (DoD) via `Get-MgContext`. A run that silently landed on `graph.microsoft.com` produced Commercial-cloud data and is not evidence.
- **Keep every artifact in-boundary.** Evidence, POA&M exports, staging folders, Log Analytics workspaces, and any dashboard must live inside your **Azure Government / GCC High** authorization boundary — never a Commercial-cloud store. This protects data residency and the ATO.
- **Do not design on unavailable features.** Re-check `gcc-high-parity-notes.md` before adding any

control or integration. Notably: the **Intune Power BI / Data-Warehouse connector** and the **ServiceNow connector** are **not available** in GCC High — use this solution's exports plus **Log Analytics** and a **manual POA&M import** instead.

Evidence retention

- **Retain the raw and the report.** Keep both the per-device Validation output *and* the finished

Evidence/POA&M exports. The raw measurement is the primary record; the report is the derived artifact.

- **Make evidence immutable where possible.** Store exports in a versioned or write-once

(immutable / hold-enabled) in-boundary location. If the destination cannot enforce immutability, restrict write access to the compliance operators and enable file versioning and backup.

- **Match retention and cadence to your ATO, not convenience.** Set the retention period and monitoring

cadence from your **FedRAMP High / DoD IL** obligation and audit cycle (R4) — commonly **monthly** continuous monitoring with maintained POA&M — not from disk space. **Verify against your assessor.**

- **Keep the full provenance chain.** Every export already carries collection timestamp, tenant,

operator, control ID, expected value, observed value, and verdict — do not strip these fields when archiving or converting formats. They are what make the record credible.

- **Never store secrets in evidence.** Evidence is configuration state, not credentials. Do not place

tokens, keys, or connection secrets in the output folder or the exports.

- **Timestamp and name consistently.** Keep the `CCAUDITG-<type>-<timestamp>` naming so files sort

chronologically and map to a run. Do not overwrite prior exports — accumulate them; the *series* is the proof of *continuous* monitoring.

- **Back up the baseline reference too.** `CCAUDITG-Baseline.json` is the expected state your evidence

is measured against; version it under change control alongside the evidence so a past verdict can be reproduced.

Cadence and drift handling

- **Measure on a fixed schedule** (at least your ATO cadence, for example monthly) plus **after any

approved baseline change** — re-baseline immediately so expected state stays accurate and does not generate false drift.

- **Treat every Drift/Missing/Non-conformant row as an event.** Open a POA&M item, remediate under

change control, and re-measure to close. Do not let drift rows accumulate unactioned.

- **Distinguish Missing from Drift.** "Missing" (baseline never applied to a newly enrolled device) is

usually a targeting/enrollment fix; "Drift" (was compliant, changed) is a remediation. Handle them differently.

- **Remediate safely.** Always run Remediation in preview (-WhatIf) first, review the exact controls,

and only `-Commit` inside a change window with a rollback recorded (G8). Never bulk-remediate authentication-impacting controls without a pilot. Remember that **update-, BIOS-, and DHA-based** controls cannot be remediated via Intune in GCC High yet — handle those out-of-band and note it.

Scoping and change control

- **Pilot every change.** Validate the method on `CCAUDITG-Pilot-Devices` before the fleet; validate any

baseline change on the pilot before re-baselining fleet-wide.

- **Keep scope explicit.** Drive measurement from named groups (`CCAUDITG-Pilot-Devices`,

`CCAUDITG-Fleet-Devices`) so "who the baseline applies to" is unambiguous and Missing is meaningful.

- **Least privilege for the operator.** Read scopes for measurement; grant write scopes only for the

remediation identity, and only when remediating.

Audit & POA&M response

- **Have a one-command evidence path.** When an assessor asks "prove control X is compliant across the

fleet on date Y," you should be able to produce it from the retained exports (or a fresh run) without manual data gathering.

- **Lead with the current export, support with the trail.** Present the latest Evidence export as the

point-in-time proof and the accumulated series as proof of *continuous* compliance over the period — the FedRAMP continuous-monitoring story.

- **Map POA&M rows to remediation records (NIST 800-53 CA-5).** For any failing control, show the POA&M

item, the remediation change ticket, and the re-measurement that closed it — this demonstrates a working control process, which is often what the assessment is really testing. Import the POA&M CSV into your FedRAMP POA&M workbook / GRC manually (no native ServiceNow connector in GCC High).

- **Show the schedule.** Evidence that measurement runs on a defined cadence (the scheduled task and its

history) is itself audit evidence of continuous monitoring.

- **Keep the method documented and reproducible.** Retain this documentation set and the script versions

used so a past result can be reproduced and the process explained to an assessor.

- **Rotate and review access.** Periodically review who can read the evidence destination and who holds

the remediation write role; remove stale access.

What to avoid

- Do not run against the **Commercial** cloud by accident — always confirm US Gov.
- Do not hand-edit evidence exports — it destroys their credibility. Re-run instead.
- Do not measure against an unapproved or stale baseline — verdicts become meaningless.
- Do not monitor a control whose enforcement mechanism is **not available in GCC High** as if Intune were enforcing it — flag it as out-of-band.
- Do not expose the output folder broadly, or store any artifact outside your Gov boundary.
- Do not skip re-measurement after remediation — closure must be proven, not assumed.

Deliverables Summary — Continuous Compliance Monitoring & Audit-Evidence Automation (Government / GCC High)

This package gives you everything needed to **monitor CIS/STIG baseline drift and automate audit evidence** for Intune + Entra endpoints **on a GCC High (US sovereign-cloud) tenant** — a repeatable operations method you keep and reuse between audits, with the correct **US Gov portals and endpoints** and the **Commercial-vs-GCC High parity gaps** called out. Below is exactly what is included and what each piece is for.

Documentation

Document	What it gives you
01-solution-overview.md	The continuous-compliance model (baseline → measure → detect drift → evidence/POA&M → remediate/track), **why GCC High is a different solution not a re-login**, a **Commercial vs GCC High — what's different** section, and where evidence comes from.
02-requirements.md	Every prerequisite (existing GCC High baseline, Intune/Entra roles + **US Gov Graph** read access, in-boundary reporting destination, evidence-retention needs, pilot scope) with Verify / Implement-if-missing.
global-requirements-security.md	Category-level prerequisites shared by all security solutions (tenant, Intune, roles, enrollment, pilot group, OS, network, change control).
03-implementation.md	Exact ordered steps using the **GCC High portals** (`intune.microsoft.us`, `entra.microsoft.us`), with **inline parity caveats** where they matter: define the monitored control set, measure current vs. baseline, detect drift, and generate audit-evidence + POA&M-ready gap exports — pilot then fleet-wide, with pass/fail gates.

Document	What it gives you
05-testing.md	Formal pass/fail gates: **correct cloud**, control measured, drift detected, evidence exported, remediation closes the gap, scheduled cadence.
06-troubleshooting.md	Symptom → cause → fix for the common issues, including **GCC-High-specific gotchas** (wrong cloud, unavailable features, US Gov consent).
07-best-practices.md	Staying in the sovereign cloud, evidence retention, and audit/POA&M response for a defensible FedRAMP High / DoD IL program.
gcc-high-parity-notes.md	The cited **Commercial-vs-GCC High parity reference** (portals/endpoints, reporting/analytics, remediation/update tooling, identity, FedRAMP/POA&M context, licensing) with Microsoft citations.
08-deliverables-summary.md	This file.
executive-summary.md	One-page business summary for decision-makers.

PowerShell scripts (Microsoft Graph SDK, US Gov endpoints)

All discovery/measurement scripts are **read-only**. The two scripts that can change state default to **preview (-WhatIf)** and require an explicit `-Commit` to act. **All default to** `-Environment USGov` (pass `-Environment USGovDoD` on a DoD/IL5 tenant).

Script	Purpose	Changes state?
CCAUDITG-Discovery.ps1	Inventory the control state of your baseline (Intune profiles, compliance policies, Security Baselines, Entra CA / auth-method policies) from **US Gov Graph**; can emit a baseline-reference template.	No — read-only
CCAUDITG-Validation.ps1	Measure per-device conformance of current state vs. the baseline reference; verdicts: Pass / Drift / Missing / Non-conformant.	No — read-only
CCAUDITG-Report.ps1	Produce the dated audit-evidence export and the POA&M-ready gap export from a validation run.	No — read-only
CCAUDITG-Remediation.ps1	Safely re-apply drifted settings to return controls to baseline.	Preview by default (-WhatIf); `-Commit` to act

Script	Purpose	Changes state?
CCAUDITG-Set-ScopeTargeting.ps1	Build/populate the pilot and fleet target groups (`CCAUDITG-Pilot-Devices`, `CCAUDITG-Fleet-Devices`).	Preview by default (-WhatIf); `-Commit` to act

Named objects you will create and reuse

- `CCAUDITG-Pilot-Devices` — Entra security group for the pilot scope.
- `CCAUDITG-Fleet-Devices` — Entra security group for fleet-wide scope.
- `CCAUDITG-Baseline.json` — the approved expected-state reference measured against.
- **Evidence exports** — `CCAUDITG-Evidence-<timestamp>.csv/json`.
- **POA&M exports** — `CCAUDITG-POAM-<timestamp>.csv`.

Outcome

When deployed, you can prove endpoint compliance **on any day**, not just deployment day — from inside your **GCC High / DoD** boundary: drift is detected automatically, audit evidence is exported on demand, and gaps arrive POA&M-ready for FedRAMP continuous monitoring — with a method and scripts you own and reuse across audit cycles.

Scope note

The specific **CIS Benchmark and DISA STIG rule IDs and official control text are deferred to CIS and DISA** (Microsoft points GCC High/DoD admins to the DISA STIGs library on `public.cyber.mil`). This solution supplies the precise **monitoring and evidence method** on Intune + Entra in the US sovereign cloud; you supply which of your deployed baseline objects encode each control. Feature availability is doc-confirmed against Microsoft's *Intune Government Service overview* — **re-verify for GCC High/DoD** before relying on any single row (see `gcc-high-parity-notes.md`).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CCAUDITG-Discovery.ps1

Purpose: Discover and inventory the Intune and Entra objects that encode your deployed CIS/STIG baseline in a US Government (GCC High / DoD) tenant: device-configuration and Settings Catalog profiles, compliance policies, Security Baselines, and Entra Conditional Access and authentication-method policies. Optionally emits a baseline-reference template (CCAUDITG-Baseline.json). Read-only — nothing is changed. Runs against Microsoft Graph US Gov; use your-tenant-id / your-account as prompted at sign-in. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Discovery.ps1 -OutputPath C:\CCAUDITG\out (add -EmitBaselineTemplate to also write a baseline template; add -Environment USGovDoD on a DoD tenant)

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Discovery.ps1 -OutputPath C:\CCAUDITG\out (add -EmitBaselineTemplate to also write a baseline template; add -Environment USGovDoD on a DoD tenant)

CCAUDITG-Remediation.ps1

Purpose: For controls a validation run flagged as Drift in a US Government (GCC High / DoD) tenant, safely re-apply the baseline by (re)assigning the baseline target group to the affected policy so drifted devices return to expected state. Runs in preview (-WhatIf) by default: it lists exactly what it would do and changes nothing. Nothing is modified until you re-run with -Commit inside a change window. Controls whose enforcement mechanism is unavailable in GCC High (update rings, BIOS/DFCI, Device Health Attestation) cannot be remediated here — handle those out-of-band. Use your-tenant-id / your-account at sign-in. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Remediation.ps1 -ValidationPath C:\CCAUDITG\out\CCAUDITG-Validation-20260701-120000.json -OutputPath C:\CCAUDITG\out (add -Commit to apply; add -Environment USGovDoD on a DoD tenant)

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Remediation.ps1 -ValidationPath C:\CCAUDITG\out\CCAUDITG-Validation-20260701-120000.json -OutputPath C:\CCAUDITG\out (add -Commit to apply; add -Environment USGovDoD on a DoD tenant)

CCAUDITG-Report.ps1

Purpose: Turn a validation run into two auditor-ready exports for a US Government (GCC High / DoD) tenant: an audit-evidence package (dated, per-control, per-device record with tenant, operator, expected, observed, and verdict) and a POA&M-ready gap list (one row per failing control with affected-device count, severity, and empty milestone/owner columns to complete for NIST 800-53 CA-5). Read-only — it consumes an existing validation file and writes exports; it queries nothing and modifies nothing. The Intune ServiceNow connector is not available in GCC High, so import the POA&M CSV into your GRC/ITSM or FedRAMP POA&M workbook manually. Use your-tenant-id / your-account values from the run. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Report.ps1 -ValidationPath C:\CCAUDITG\out\CCAUDITG-Validation-20260701-120000.json -OutputPath C:\CCAUDITG\out

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Report.ps1 -ValidationPath C:\CCAUDITG\out\CCAUDITG-Validation-20260701-120000.json -OutputPath C:\CCAUDITG\out

CCAUDITG-Set-ScopeTargeting.ps1

Purpose: Create and populate the Entra security groups that define monitoring scope in a US Government (GCC High / DoD) tenant — the pilot group (CCAUDITG-Pilot-Devices) and the fleet group (CCAUDITG-Fleet-Devices). Runs in preview (-WhatIf) by default: it shows the group it would create and the members it would add, and changes nothing. Nothing is created or modified until you re-run with -Commit. Use your-tenant-id / your-account at sign-in. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Set-ScopeTargeting.ps1 -GroupName CCAUDITG-Pilot-Devices -OutputPath C:\CCAUDITG\out (add -Commit to create/populate; add -Environment USGovDoD on a DoD tenant)

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Set-ScopeTargeting.ps1 -GroupName CCAUDITG-Pilot-Devices -OutputPath C:\CCAUDITG\out (add -Commit to create/populate; add -Environment USGovDoD on a DoD tenant)

CCAUDITG-Validation.ps1

Purpose: Measure the current state of each monitored control against the approved baseline reference (CCAUDITG-Baseline.json) for every device in a target group in a US Government (GCC High / DoD) tenant, and assign a conformance verdict per device per control: Pass, Drift, Missing, or Non-conformant. This is how baseline drift is detected. Read-only — nothing is changed. Runs against Microsoft Graph US Gov; supply your-tenant-id / your-account at sign-in. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Validation.ps1 -BaselinePath .\CCAUDITG-Baseline.json -GroupName CCAUDITG-Pilot-Devices -OutputPath C:\CCAUDITG\out (add -Environment USGovDoD on a DoD tenant)

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDITG-Validation.ps1 -BaselinePath .\CCAUDITG-Baseline.json -GroupName CCAUDITG-Pilot-Devices -OutputPath C:\CCAUDITG\out (add -Environment USGovDoD on a DoD tenant)